

Lab 3

```
#include <stdio.h>
```

```
// 1. Linear Search
```

```
int search(int a[3][3], int m, int n) {  
    int key = 4;  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            if (a[i][j] == key) {  
                printf("\nElement %d found at [%d][%d] index", key, i, j);  
                return key;  
            }  
        }  
    }  
    printf("\nElement %d not found", key);  
    return -1;  
}
```

```
// 2. SUM
```

```
void sum(int a[3][3], int m, int n) {  
    int sum = 0;  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            sum += a[i][j];  
        }  
    }  
    printf("\nsum=%d", sum);  
}
```

```
// 3. Min & Max
```

```
void Minmax(int a[3][3], int m, int n) {
```

```

int min = a[0][0];

int max = a[0][0];

for (int i = 0; i < m; i++) {

    for (int j = 0; j < n; j++) {

        if (a[i][j] < min) min = a[i][j];

        if (a[i][j] > max) max = a[i][j];

    }

}

printf("\nMIN =%d\n MAX =%d", min, max);

}

```

// 4. Hadamard Multiplication

```

void multiply(int a[3][3], int b[3][3], int c[3][3], int m, int n) {

    for (int i = 0; i < m; i++) {

        for (int j = 0; j < n; j++) {

            c[i][j] = a[i][j] * b[i][j];

        }

    }

    printf("\nProduct is\n");

    for (int i = 0; i < m; i++) {

        for (int j = 0; j < n; j++) {

            printf("%d\t", c[i][j]);

        }

        printf("\n");

    }

}

```

// 5. Printing primary and secondary diagonal

```

void diagonal(int a[3][3], int m, int n) {

    printf("\nDiagonals: ");

    for (int i = 0; i < m; i++) {

```

```

    for (int j = 0; j < n; j++) {
        if (i == j || i + j == n - 1) {
            printf("%d ", a[i][j]);
        }
    }
}
}
}

```

// 6. Sum of upper triangular matrix

```

void sumu(int a[3][3], int m, int n) {
    int sumu = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (i <= j) {
                sumu += a[i][j];
            }
        }
    }
    printf("\nSum of upper triangle = %d", sumu);
}

```

// 7. Sum of Lower triangular matrix

```

void suml(int a[3][3], int m, int n) {
    int suml = 0;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (i >= j) {
                suml += a[i][j];
            }
        }
    }
}

```

```
printf("\nSum lower triangle = %d", suml);  
}
```

// 8. Transpose

```
void transpose(int a[3][3], int T[3][3], int m, int n) {  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            T[i][j] = a[j][i];  
        }  
    }  
    printf("\nTranspose\n");  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            printf("%d\t", T[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
int main() {  
    int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};  
    int b[3][3] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};  
    int c[3][3];  
    int T[3][3];  
  
    search(a, 3, 3);  
    sum(a, 3, 3);  
    Minmax(a, 3, 3);  
    multiply(a, b, c, 3, 3);  
    diagonal(a, 3, 3);  
    sumu(a, 3, 3);  
}
```

```
suml(a, 3, 3);
```

```
transpose(a, T, 3, 3);
```

```
return 0;
```

```
}
```

The screenshot shows the HackerRank interface for the 'Diagonal Difference' problem. The problem description on the left explains that given a square matrix, the task is to calculate the absolute difference between the sums of its diagonals. An example matrix is provided: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 9 & 8 & 9 \end{bmatrix}$. The left-to-right diagonal sum is 15, and the right-to-left diagonal sum is 17, resulting in an absolute difference of 2. The function signature is `int diagonalDifference(int arr_rows, int arr_columns, int** arr)`. The solution code on the right implements this function by calculating the sums of both diagonals and returning their absolute difference.

```
20  * Complete the 'diagonalDifference' function below.
21  *
22  * The function is expected to return an INTEGER.
23  * The function accepts 2D_INTEGER_ARRAY arr as parameter.
24  */
25
26  int diagonalDifference(int arr_rows, int arr_columns, int** arr) {
27      int pdsun=0;
28      int sdsun=0;
29      for (int i=0;i<arr_rows;i++){
30          for (int j=0;j<arr_columns;j++){
31              if (i==j){
32                  pdsun+=arr[i][j];
33              }if(i+j==arr_rows-1){
34                  sdsun+=arr[i][j];
35              }
36          }
37      }
38      int diff = pdsun-sdsun;
39      if (diff < 0) diff = -diff;
40
41      return diff;
42
43
44
45  }
```

The screenshot shows the HackerRank interface for the 'Plus Minus' problem. The problem description on the left states that given an array of integers, the task is to calculate the ratios of positive, negative, and zero elements. An example array is provided: `arr = [1, 1, 0, -1, -1]`. The ratios are 0.4, 0.4, and 0.2 respectively. The function signature is `void plusMinus(int arr_count, int* a)`. The solution code on the right implements this function by counting the positive, negative, and zero elements and then calculating their ratios.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  char* readLine();
6  char* ltrim(char*);
7  char* rtrim(char*);
8  char** split_string(char*);
9
10 int parse_int(char*);
11
12 /*
13  * Complete the 'plusMinus' function below.
14  * The function accepts INTEGER_ARRAY arr as parameter.
15  */
16
17 void plusMinus(int arr_count, int* a) {
18     int pcount=0;
19     int ncount=0;
20     int zcount=0;
21     for (int i=0;i<arr_count;i++){
22         if(a[i]>0) pcount++;
23         else if(a[i]<0) ncount++;
24         else zcount++;
25     }
26     float pr = pcount*1.0/arr_count;
27     float nr = ncount*1.0/arr_count;
28     float zr = zcount*1.0/arr_count;
29     printf("%f\n",pr);
30     printf("%f\n",nr);
31     printf("%f\n",zr);
32 }
33
34 int main()
35 {
36     int arr_count;
37     int* arr;
38     scanf("%d", &arr_count);
39     arr = (int*) malloc(arr_count * sizeof(int));
40     for (int i = 0; i < arr_count; i++) {
41         scanf("%d", &arr[i]);
42     }
43     plusMinus(arr_count, arr);
44 }
```